

PRACTICAL – 3

'''

Implement Greedy search algorithm for any of the following application:

I. Selection Sort

II. Minimum Spanning Tree

III. Single-Source Shortest Path Problem

IV. Job Scheduling Problem

V. Prim's Minimal Spanning Tree Algorithm

VI. Kruskal's Minimal Spanning Tree Algorithm

VII. Dijkstra's Minimal Spanning Tree Algorithm

'''

```
import heapq
```

```
def prims(graph, start):
```

```
    visited = set()
```

```
    min_heap = [(0, start, -1)] # (weight, current_node, parent)
```

```
    total_cost = 0
```

```
    mst_edges = []
```

```
    while min_heap:
```

```
        weight, node, parent = heapq.heappop(min_heap)
```

```
        if node in visited:
```

```
            continue
```

```
        visited.add(node)
```

```
        total_cost += weight
```

```
        if parent != -1:
```

```
            mst_edges.append((parent, node, weight))
```

```
        for neighbor, wt in graph[node]:
```

```
            if neighbor not in visited:
```

```
                heapq.heappush(min_heap, (wt, neighbor, node))
```

```
    return mst_edges, total_cost
```

```
# ----- MAIN PROGRAM -----
```

```
graph = {}
```

```
n = int(input("Enter number of edges: "))
```

```
print("Enter edges (u v weight):")
```

```
for _ in range(n):
```

```
    u, v, w = input().split()
```

```
w = int(w)
```

```
graph.setdefault(u, []).append((v, w))
```

```
graph.setdefault(v, []).append((u, w)) # undirected graph
```

```
start = input("Enter starting vertex: ")
```

```
mst, cost = prims(graph, start)
```

```
print("\nEdges in MST:")
```

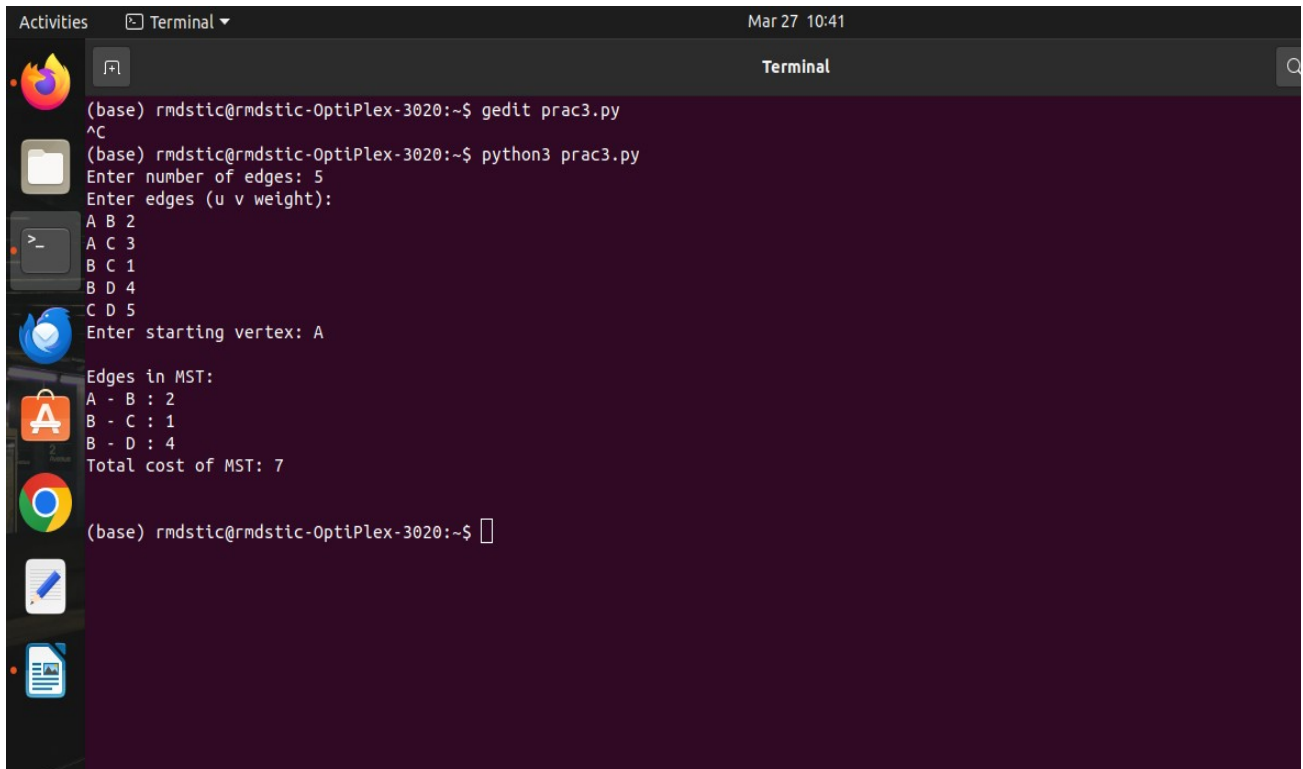
```
for u, v, w in mst:
```

```
    print(f"{u} - {v} : {w}")
```

```
print("Total cost of MST:", cost)
```

```
print("\n")
```

OUTPUT



The image shows a terminal window titled "Terminal" with a dark background. The window displays the execution of a Python script named "prac3.py". The user enters the command "python3 prac3.py" and the script prompts for the number of edges (5) and the edges themselves (A B 2, A C 3, B C 1, B D 4, C D 5). The script then prompts for the starting vertex (A) and outputs the edges in the MST (A - B : 2, B - C : 1, B - D : 4) and the total cost of the MST (7).

```
(base) rmdstic@rmdstic-OptiPlex-3020:~$ gedit prac3.py
^C
(base) rmdstic@rmdstic-OptiPlex-3020:~$ python3 prac3.py
Enter number of edges: 5
Enter edges (u v weight):
A B 2
A C 3
B C 1
B D 4
C D 5
Enter starting vertex: A

Edges in MST:
A - B : 2
B - C : 1
B - D : 4
Total cost of MST: 7

(base) rmdstic@rmdstic-OptiPlex-3020:~$
```